Proteus Automation System
Product Design Document

Date			
Original Date:	04/01/21	**Document Version**	0.1
Last Revision Date:			
		Author Information	
Product Name:	Proteus Automation	Author:	Kavitha Patchamalai
		Title:	Product Management
		E-mail:	experis.automation@dell.com

SECURITY WARNING

The information contained herein is proprietary to Dell Inc. and must not be disclosed to non-Dell personnel. The recipient of this document, by its retention and use, agrees to protect the information contained herein.

Readers are advised that this project may be subject to the terms of a non-disclosure agreement.

DO NOT DISCLOSE ANY OF THIS INFORMATION WITHOUT OBTAINING PERMISSION FROM THE MANAGEMENT RESPONSIBLE FOR THIS

1 Document Revision History

The revision history shows the history for this Product Design Document and provides descriptions of changes made.

Date	Version	Section/Page	Update Description	Contact Name
03/12/21	1	All	Initial Draft	April Jewell

2 Contributors

The content of this document has been authored with the combined input of the following group of key individuals.

Name	Role	Area
Gurvinder Bindra	Solutions Architect / Workflow Engineer	

3 Document Approval History

The section tracks the review and approval of this Product Design Document.

Name	Email	Role	Responsibility
April Jewel	April_Jewell@Dellteam.com	Project Manager	Approver

Table of Contents

1 Document Revision History 2
2 Contributors 2
3 Document Approval History 2
1 Overview 4
1.1 Proteus Automation System Goal 4
1.2 Project Background 4
1.3 Project Scope statement: 4
2 Roles and Responsibilities 5
3 Current System- Dell's Existing Workflow 6
4 Proposed System- Proteus Automation System Workflow 6
5 Acceptance Criteria 6
6 SN001 Proposed 7
6.1 Rollback to Original Configuration 7
7 Design Constrains 8
8 Development Tools, Related Software and Hardware 8
9 High Level Design 9
9.1 Proteus Automation System Components 9
9.2 Sawmill General Flow ..... **Error! Bookmark not defined.**
10 UI Walkthrough 11
11 Standard Changes- Fully Automated 11
11.1 Create a Request 12
11.2 Ansible Tower Schedules and Executes the Job 12
11.3 Change Request Completed in ServiceNow 13
11.4 Rollback Completed Successfully 13
11.5 Review Original and Changed Config with Rollback ID 14
11.6 Review Closed Rollback Ticket 15
12 Standard Changes with Manual Intervention 15
12.1 Create a Request 15
12.2 Ansible Tower Schedules and Executes the Job 16
12.3 Review Request and Approve RFC 16
12.4 Rollback to Original Configuration 17
12.5 Review Original and Changed Config with Rollback ID 17
13 Standard Changes with Specific Schedule 18
13.1 Create a Change Request 18
13.2 Ansible Tower Schedules and Executes the Job 19
13.3 Review Request and Approve RFC 19
13.4 Rollback to Original Configuration 20
13.5 Rollback ID for Reviewing Original and Changed Config 20
14 System Security and Integrity Controls 21
14.1 Role-based AD Authentication 21
14.2 Security Testing 21
15 Appendix 21
15.1 Project References ..... **Error! Bookmark not defined.**
15.2 GitLab Code Repositories ..... **Error! Bookmark not defined.**

1 Overview

The Net Proteus application is created to automate various tasks in Load balancing, Networking, and DNS administration that are currently handled mostly manually by various engineers/administrators with minimal levels of automation.
Proteus Automation system is designed to provide the following functionalities:

Feature Name	Description
SAWMILL	Primary Backend Engine that accepts parameters from UI and integrates Gitlab, ServiceNow and Ansible Tower using several microservices and MongoDB.
SN001	SAWMILL's capability of executing a Job end to end while creating, updating, and closing relevant RFC tickets.
LB001	Ability to identify existing server pools for the existing VIPs and modifying the server members of the given server pools.
NW001	Ability to change SNMP access list for the given network devices.
UI001	Net Proteus' user interface and its essential features.

1.1 Proteus Automation System Goal

- Create, update and close RFC tickets by executing GitLab CI/CD pipeline through Ansible Tower.
- Configure various systems as per Playbooks created in Ansible Tower.
- Create various interfaces to integrate systems. Example., Microservice to identify status of Job in AWX and update in ServiceNow through Gitlab CI/CD.
- Rollback to original configuration in case of failure.

1.2 Project Background

- To scale, expand and improve Dell's internal and/or cloud hosting environments through design, integration, and deployment of additional Network Management Services, stack components and related technologies.
- To analyze, plan, and implement IaaS services related to transforming Network, DNS and Load Balancing product lines to automated API based services.

1.3 Project Scope statement:

- Identify and automate the requests across Networking for executing Load Balancing requests, DNS requests, and Network requests.
- Create service catalog request items, initially in ServiceNow, for the adds and changes, along with the supporting code, workflows, and integrations necessary to automate approval and implementation processes for in-scope items including:
    a) Switches and Routers
    b) Load Balancing

c) DNS
d) IPAM
e) DHCP
f) F5
g) Bluecat
h) Infoblox
i) Potential future products and platforms such as VMWare AVI

- Use Ansible Tower and other Dell approved tools, build back-end automation to support the intake, review, and approval of both Greenfield Provisioning and Brownfield Conversion, including bare metal to virtual transition as appropriate, and reconfiguration deployment for:
  a) Network: Switches, Routers, Tools and Wireless Controllers
  b) Full Prox Load Balancing (i.e. F5, LTM, Ingress)
  c) GSLB (i.e. GTM)
  d) DDI (i.e. Bluecat and future Infoblox)
- Build and publish APIs for integration of the various systems to support automation and delivery as described above.
- Create Interface for Access to Consumption, Billing, and Metering information and facilitate both reporting through dashboards and exchange of data with other systems for use by Finance in the chargeback process.

2 Roles and Responsibilities

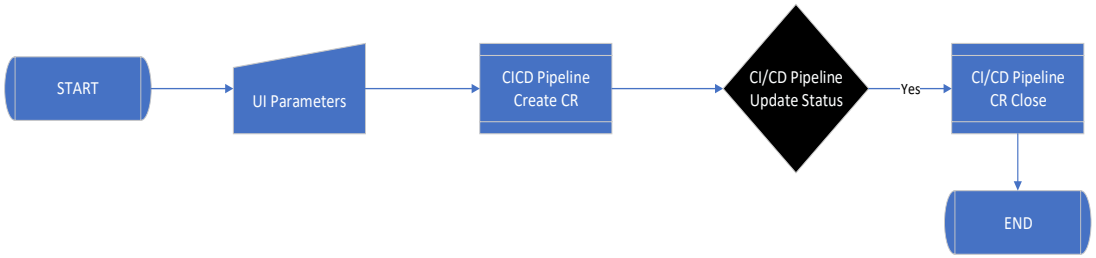
This matrix serves as the list of point of contact for issues and concerns related to Proteus System Design:

Name	Primary Role	Location	Contact Information
Matthew Randol	Account Director	U.S., Texas	
April Jewell	Project Manager	U.S. Georgia	678-592-0539 April_Jewell@Dellteam.com
Gurvinder Bindra	Solutions Architect / Workflow Engineer	U.S. California	+1 (805) 202 6377 gurvinder_bindra@dellteam.com
Ashwini Kumar Kalyanpur	UI Front end Developer	India	919880059175 ashwini_kalyanpur@dellteam.com
Sarath Chintapatla	Middleware Developer	India	
Srikanth Palasani	DevOps Automation Engineer	India	Srikanth_Palasani@Dellteam.com
Mridula Krishnamurthy	DevOps Engineer	India	+91 9740858081 Mridula_krishnamurth@dellteam.com

Raja Kumar Annam	QA Testing	India	Kumar, Annam - Dell Team Annam_Kumar@Dellteam.com>

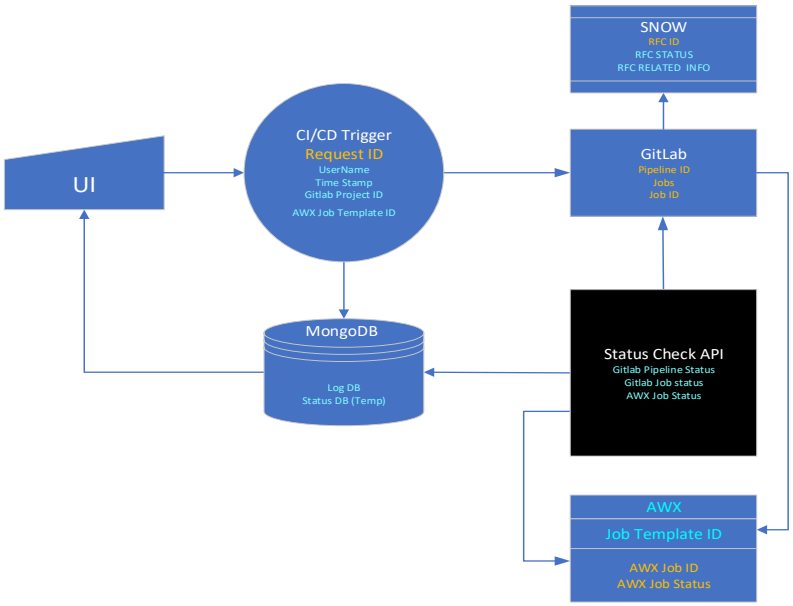
3 Current System- Dell's Existing Workflow

The existing Dell's workflow is manual where the user works on the CR step by step and manually updates the CR status in the ServiceNow application.



4 Proposed System- Proteus Automation System Workflow

The proposed workflow is automated where Proteus accepts parameters from the UI, integrates GitLab, ServiceNow and Ansible Tower using several microservices and MongoDB; and executes the CI/CD pipeline to create, update and close complete a CR.

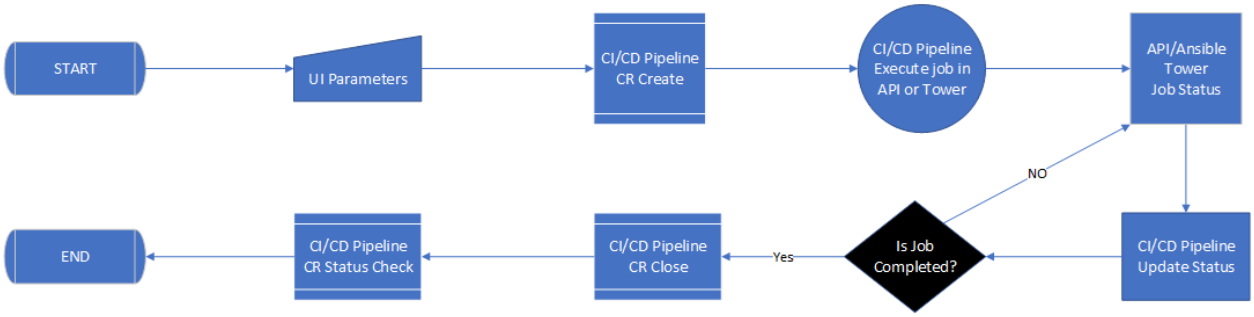


5 Acceptance Criteria

- Ability to create change request (CR).
- Ability to schedule change through UI
- Ability to monitor execution within SLA timeframe.
- Ability to query Ansible Tower / API through CI/CD pipeline.

- Ability to update CR with status of Ansible tower job / API Calls.
- Ability to identify failures.
- Ability to cause rollback in case of failure.
- Ability to close CR with appropriate status of Ansible tower.

6 SN001 Proposed



When the user creates a CR through the UI, Proteus executes the following steps with CI/CD pipeline:

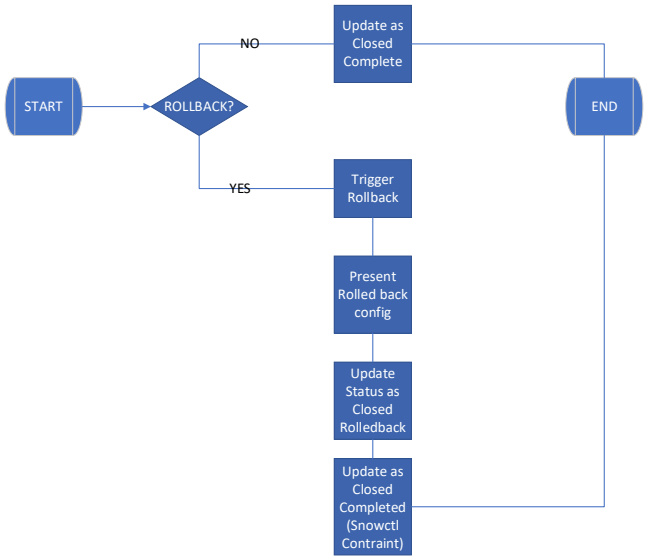
1. Schedules a Job with Ansible Tower.
2. Executes the Job with Ansible Tower.
3. Verifies Ansible Tower and GitLab for Job Status.
4. Retrieves information from ServiceNow, GitLab, Ansible Tower and stores in a central repository.
5. Updates the RFC ticket in ServiceNow.
6. Closes the RFC ticket in ServiceNow.
7. In case of failure, identifies it and provides the option to rollback.

6.1 Rollback to Original Configuration

Proteus provides the option to revert to original configuration. The user can download and view these configuration files:

- Original Configuration
- Changed Configuration
- Rolled Back Configuration

Proteus displays the rollback button only after the RFC is closed in ServiceNow and creates a separate Rollback ID (AWX Job ID) for Rollback operation.



7 Design Constrains

The main constraint associated with Proteus Automation design is that the technical team has:

1. **No direct access to Dell Technologies-ServiceNow (SNOW)**
   - Due to this issue, the application design is dependent on using SNOWCli and the GitLab Pipeline. This interaction has been noted to cause delays in deployment specifically with respect to the performance of GitLab and the availability of Runners. The Runners situation has been addressed by using dedicated VMs as Runners instead of the shared VMs provided by GitLab. It needs to be noted that this adds additional peripheral costs and administrative /maintenance overheads. The application's requirement to create, modify, update and close Change Tickets (RFC) needs either direct SNOW API access or pipeline access through GitLab.

2. **No direct access to Change Management Database**
   - Similar to the Service Now API access, the Change Management Database does not permit direct access to the Database. It has been observed and widely recognized that the Database itself is not optimally configured as of writing of this document. The access to the contents of the Database are provided through a secondary dump which is expected to not be in sync with the actual database. With respect to the currently developed features, the CMDB is used to identify the CI ids (SysIDs for the impacted systems and devices for which the changes are being implemented.

8 Development Tools, Related Software and Hardware

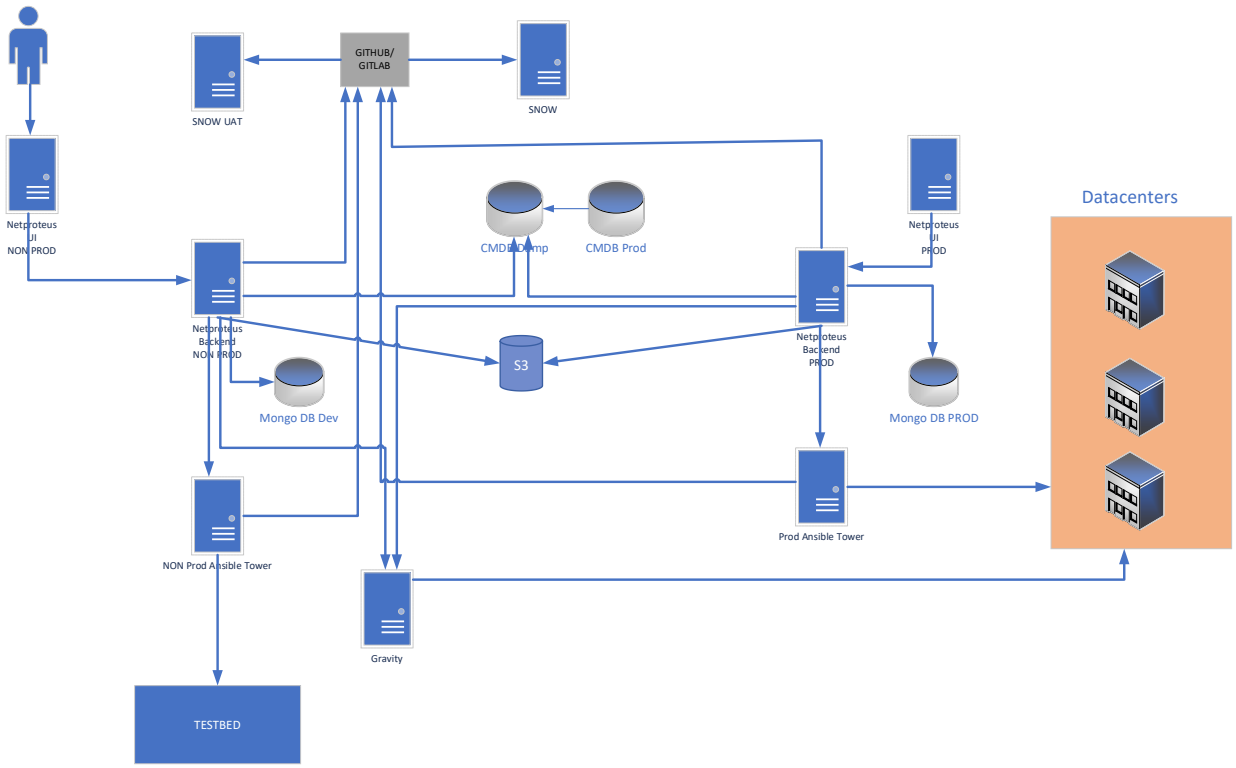
The development tools used include hardware, operating systems, and tools for program development, source control, general design, database design, debugging, defect tracking, and testing:

Infrastructure	UI	Backend	Testing	Devops	Testbed
Dell Digital Cloud PKS	Angular JS	Python 3x	pytest	Ansible	F5 Devices
Gitlab (Code Management)	Html/CSS	Flask	postman	Gitlab CI/CD	IOS Devices / Images
Docker	Bootstrap	MongoDB	Dell Customized tools	Hashicorp Vault	DELL OS 6
Service Now (inc CMDB)			SaaS Labs Virtual env		DELL OS 9
Dell Digital Cloud VMs			Tox		DELL 10 Virtual
Dell Digital Cloud Object Storage (S3)			Coverage		ESXi Hosts and Storage
			SonarCube		Bluecat Testbed - Virtual
			Blackduck		Infoblox Testbed
			Checkmarx		

9 High Level Design

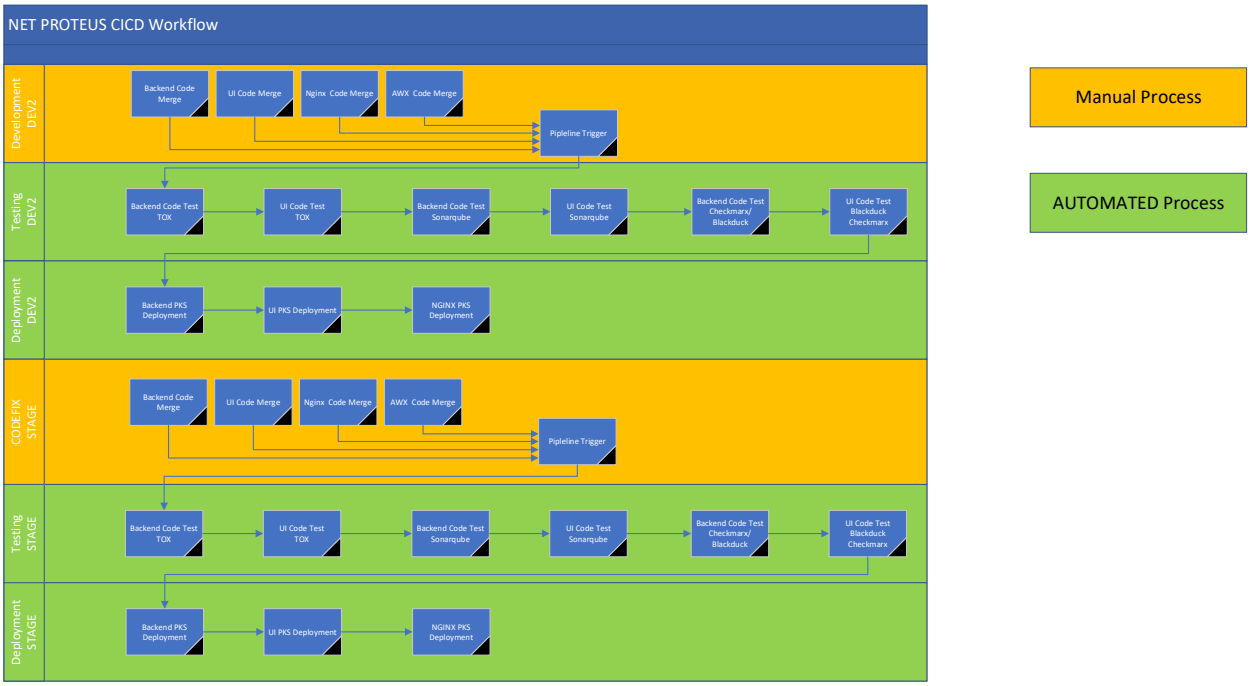
This section provides a high-level overview of how the functionality and responsibilities of the system are partitioned and then assigned to subsystems or components. The description includes diagrams displaying interactions between the subsystems, and diagrams displaying the data flow in the application.

9.1 Proteus Automation System Components



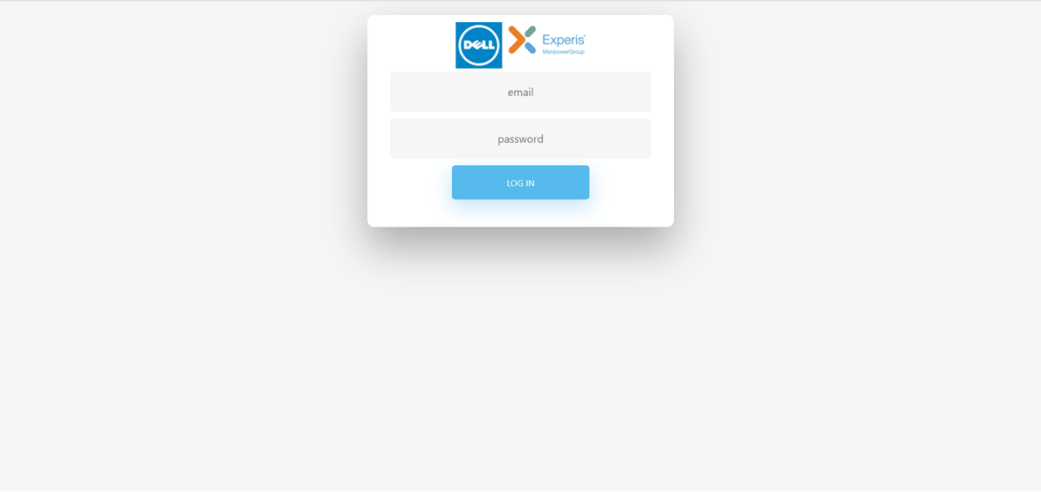
The proposed workflow is fully automated where Proteus accepts parameters from the UI, integrates GitLab, ServiceNow and Ansible Tower using several microservices and MongoDB; and executes the CI/CD pipeline to create, update and close complete a CR.

10 Application CI/CD Workflow



11 UI Walkthrough

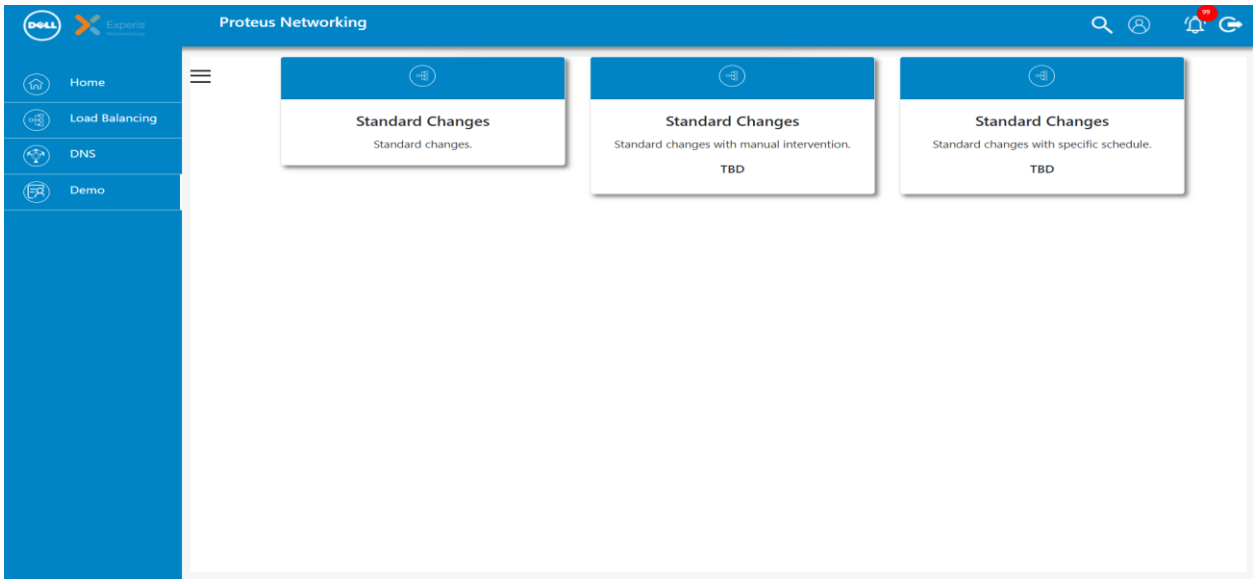
Proteus restricts access based on the user's role (RBAC) within Dell Technologies.



12 Standard Changes- Fully Automated

The standard changes functionality is fully automated that executes the CR with manual intervention if needed/desired.

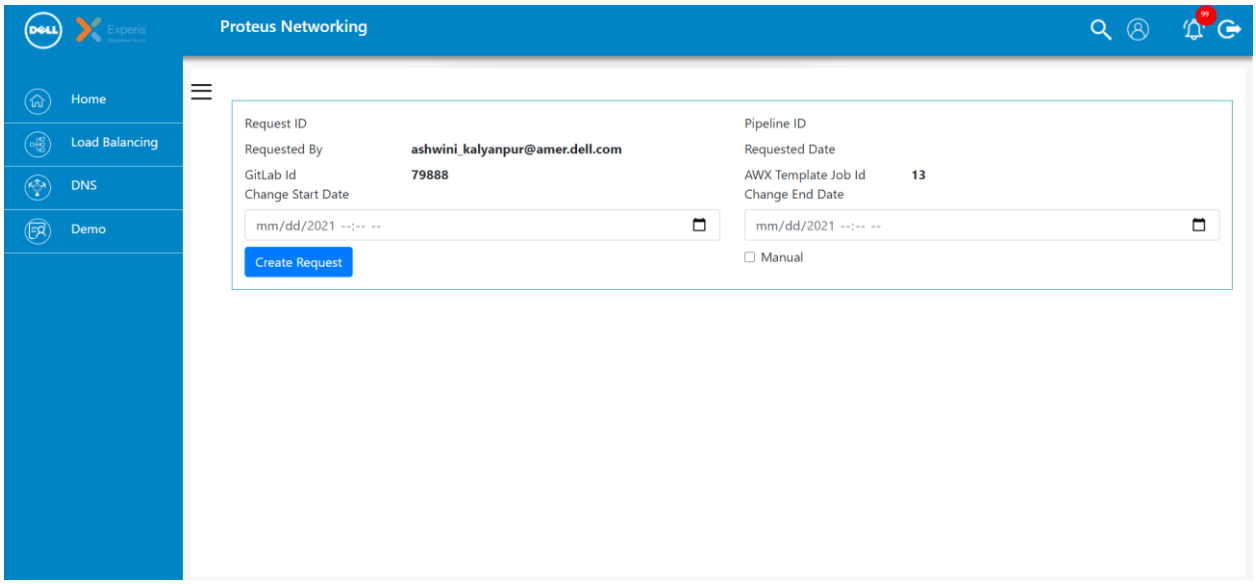
The backend operations are displayed on the UI for demo purpose.



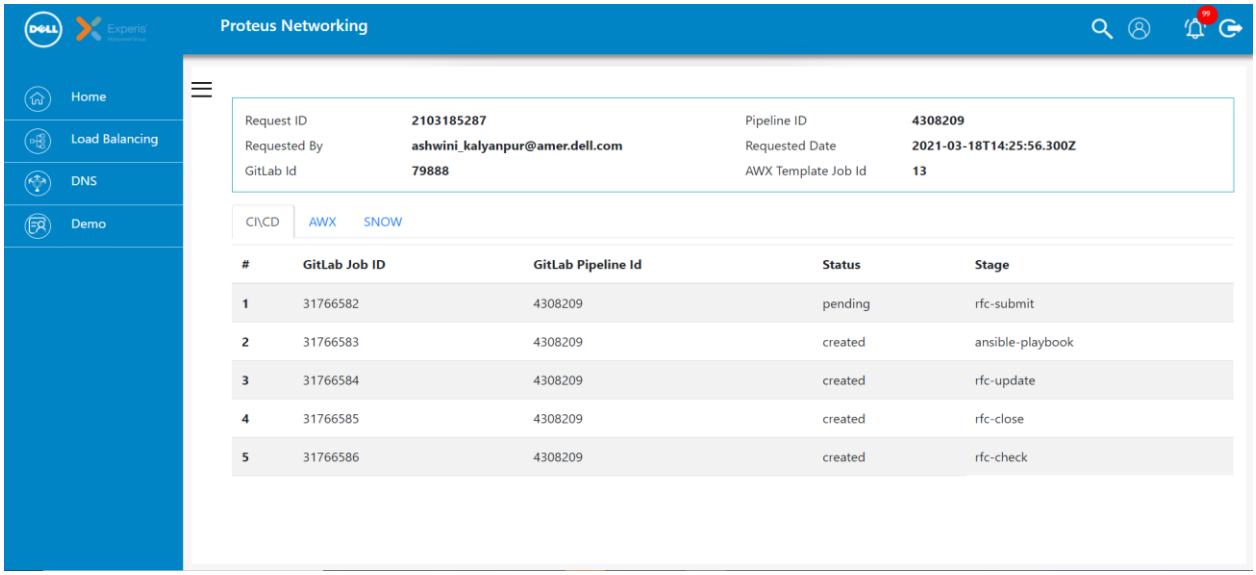
Proteus Automation System provides the following options to close complete a Change Request:

Option	Description
STANDARD CHANGES- FULLY AUTOMATED	This functionality is fully automated with no manual intervention.
STANDARD CHANGES WITH MANUAL INTERVENTION	This functionality allows the user to manually review and then approve the CR.
STANDARD CHANGES WITH SPECIFIC SCHEDULE	This functionality allows the user to execute standard change(s) with specific schedule.

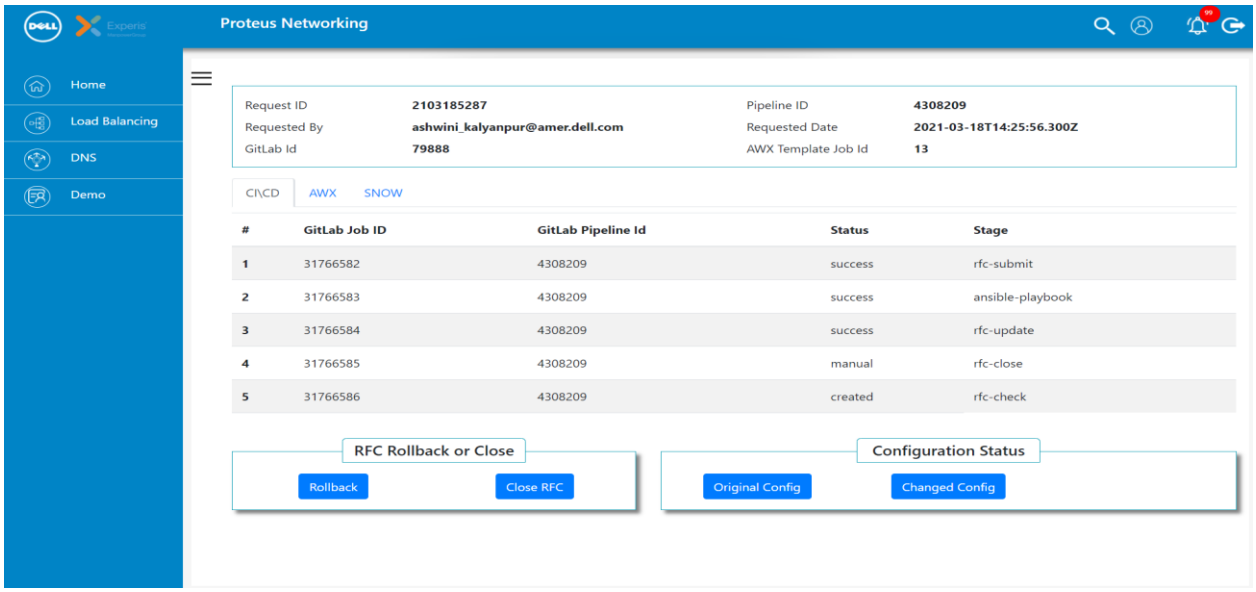
12.1 Create a Request



12.2 Ansible Tower Schedules and Executes the Job

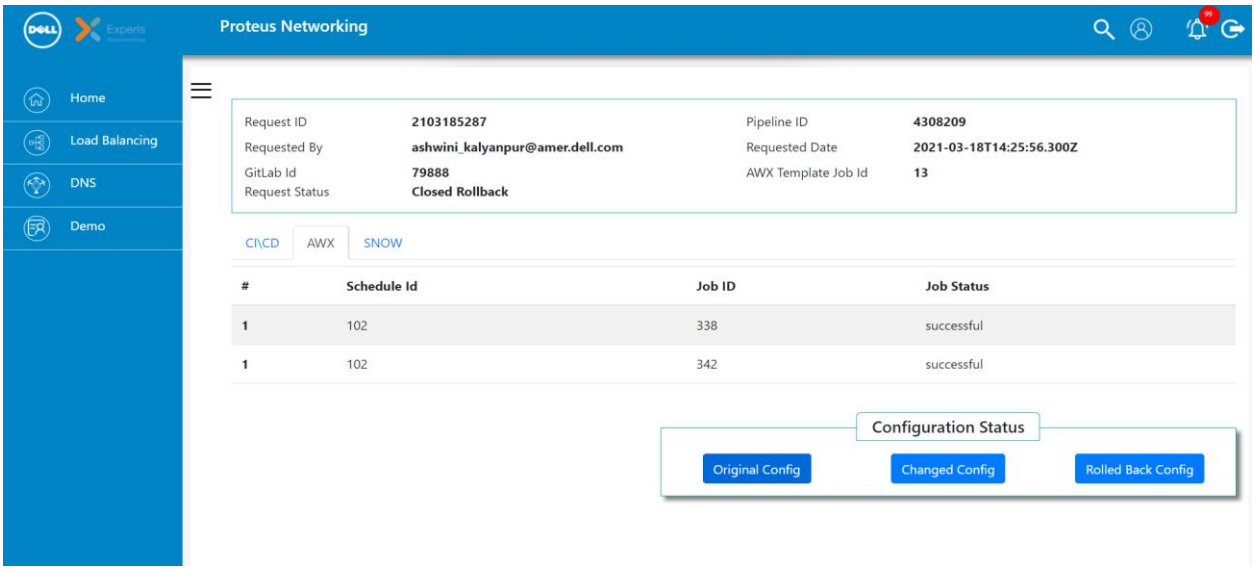


12.3 Change Request Completed in ServiceNow



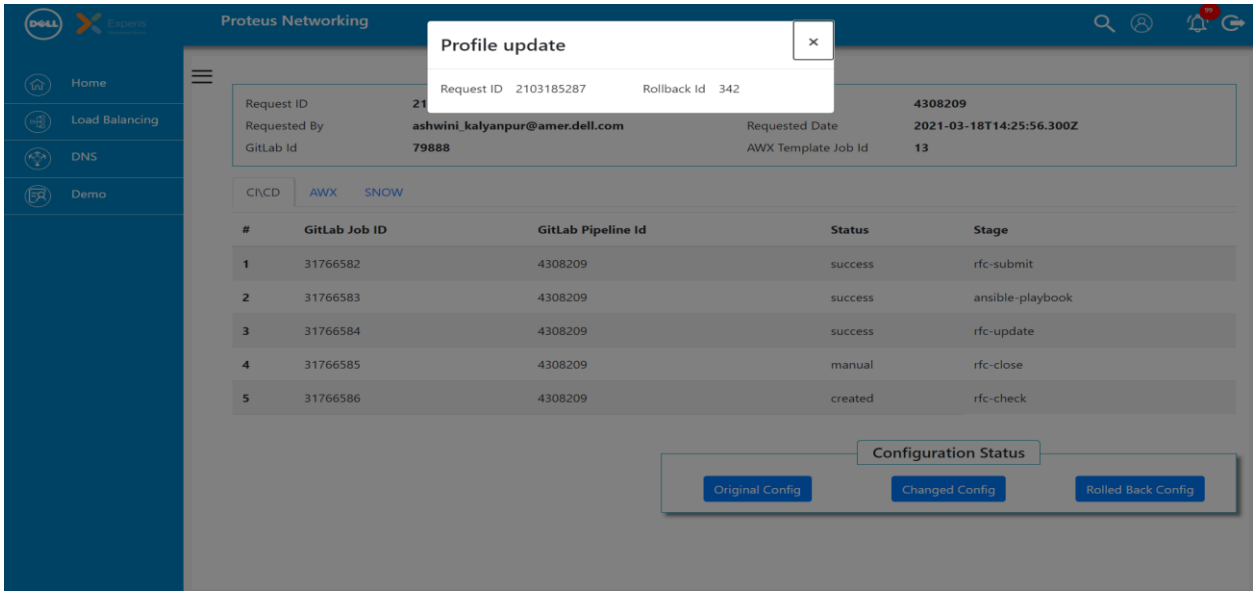
* Rollback Button appears after the RFC is Closed Complete in ServiceNow.

12.4 Rollback Completed Successfully

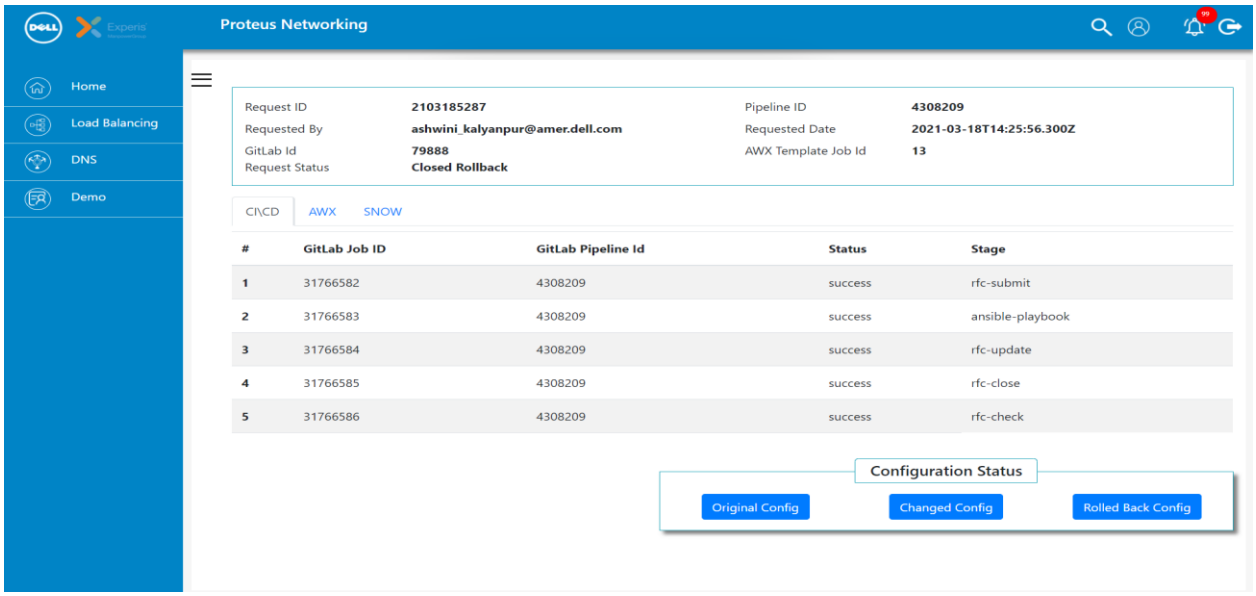


*Proteus Automation system generates a different Rollback ID (AWX Job ID) for the Closed Rollback Status.

12.5 Review Original and Changed Config with Rollback ID

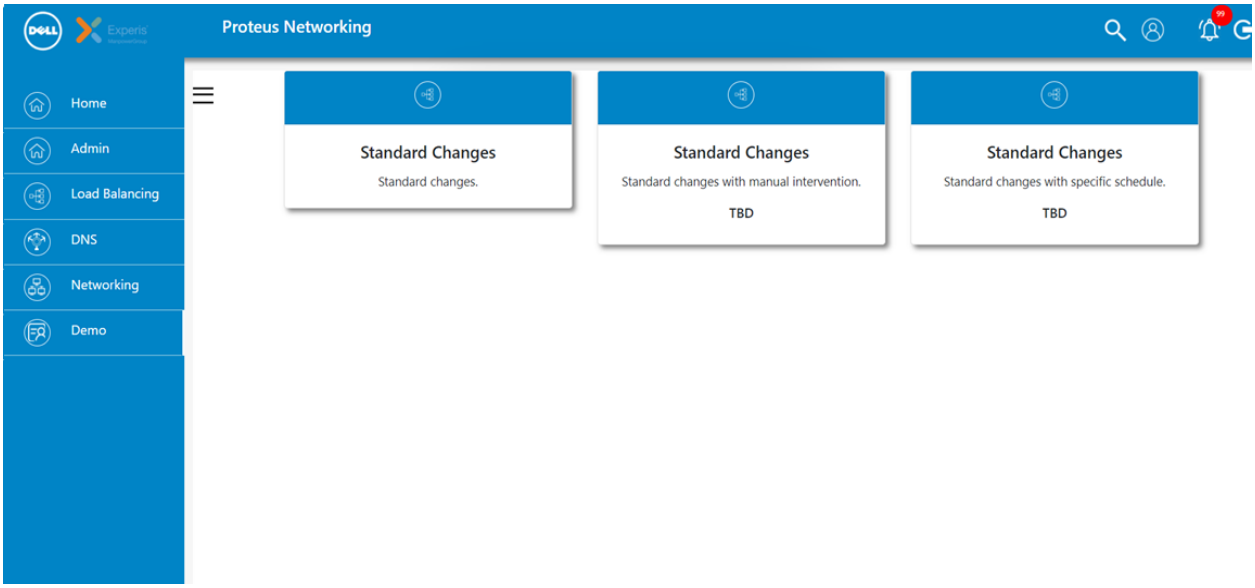


12.6 Review Closed Rollback Ticket

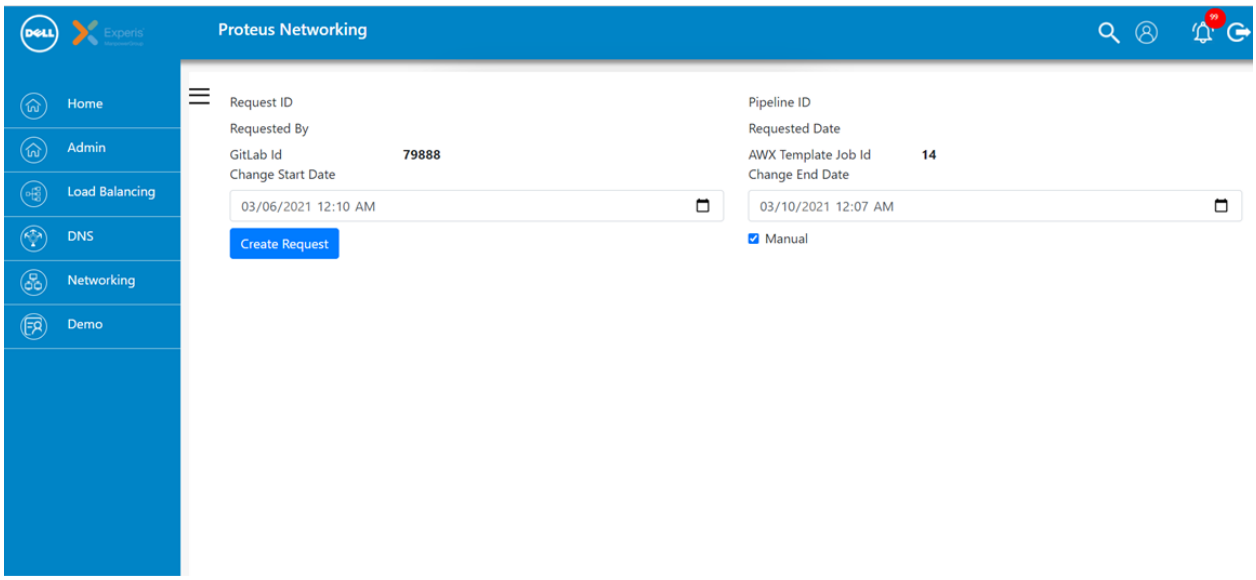


13 Standard Changes with Manual Intervention

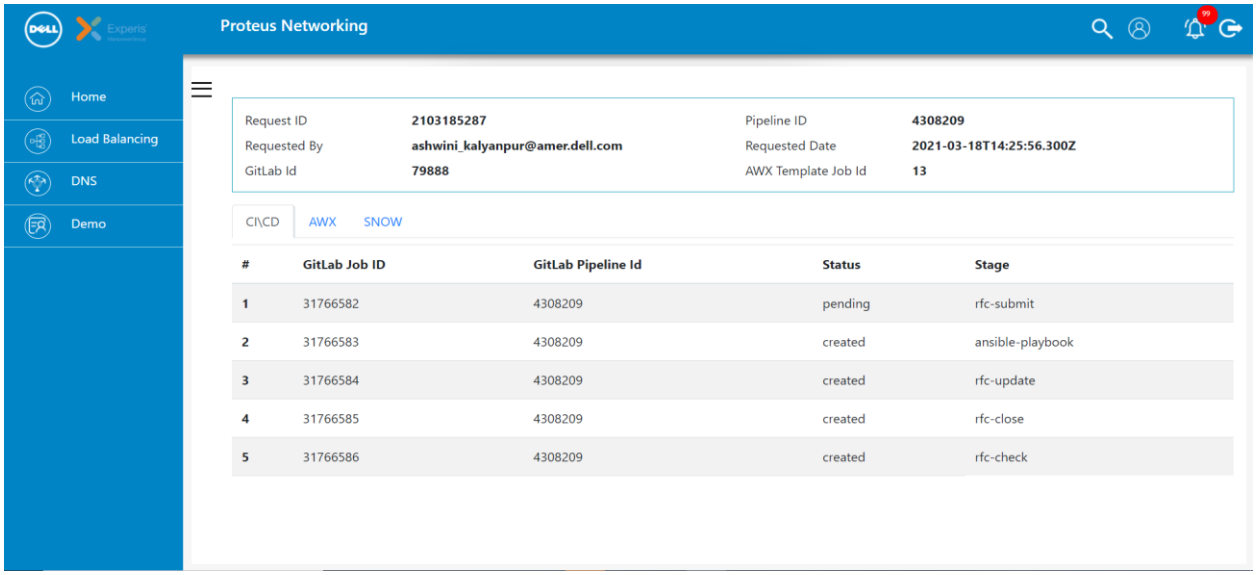
The standard change with manual intervention allows the user to review and then approve the CR workflow.



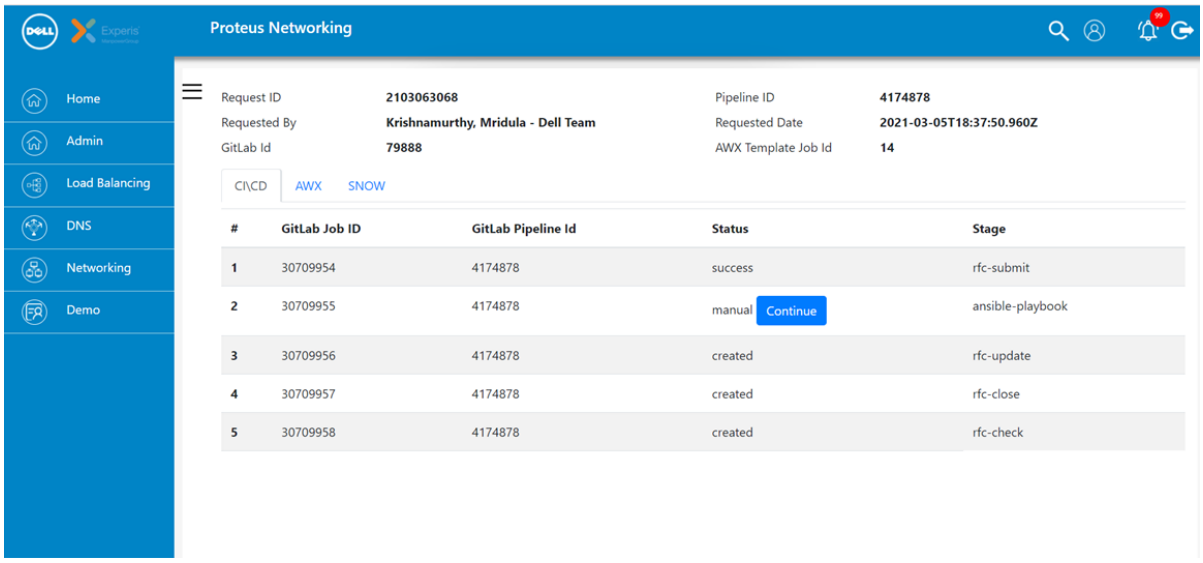
13.1 Create a Request



13.2 Ansible Tower Schedules and Executes the Job

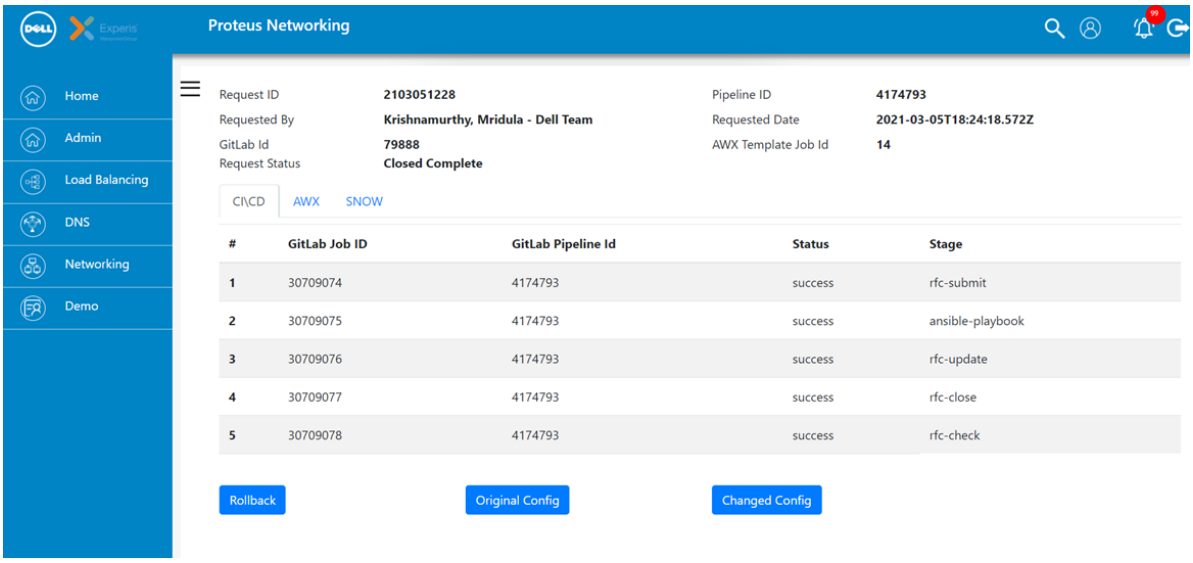


13.3 Review Request and Approve RFC

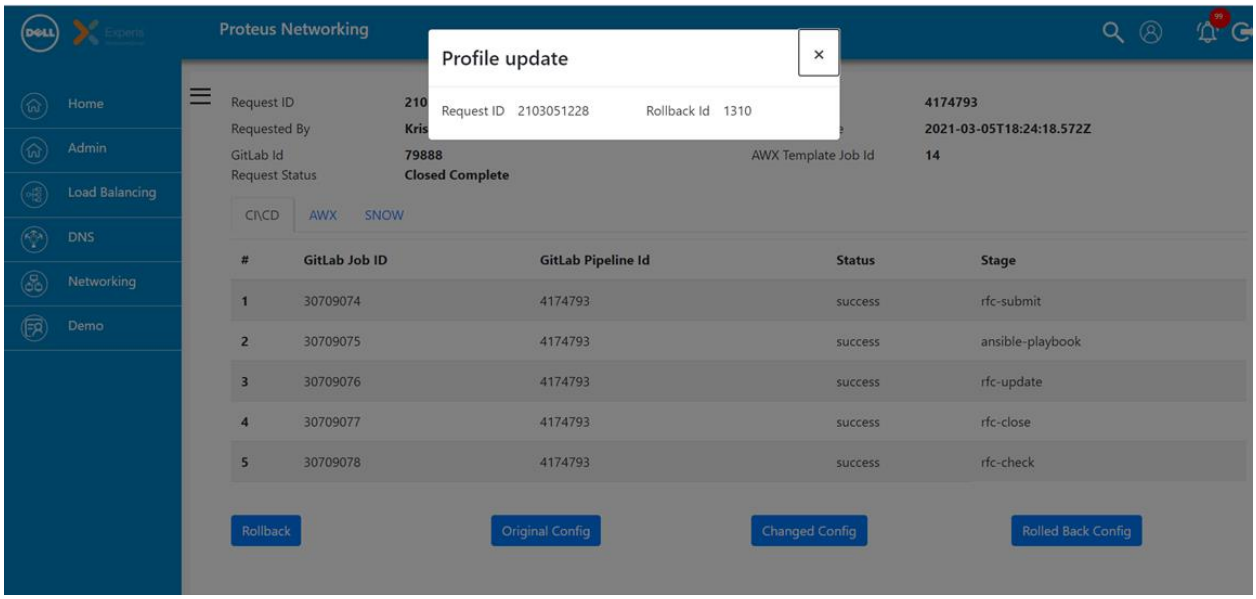


* Rollback Button Appears after the RFC is Closed Complete in ServiceNow.

13.4 Rollback to Original Configuration

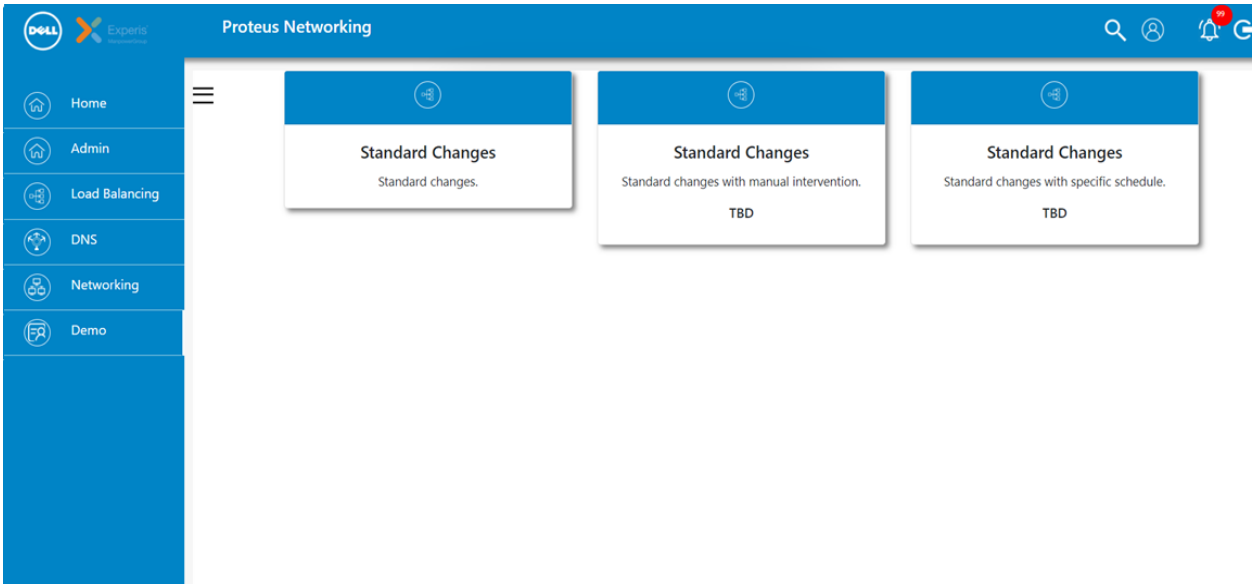


13.5 Review Original and Changed Config with Rollback ID

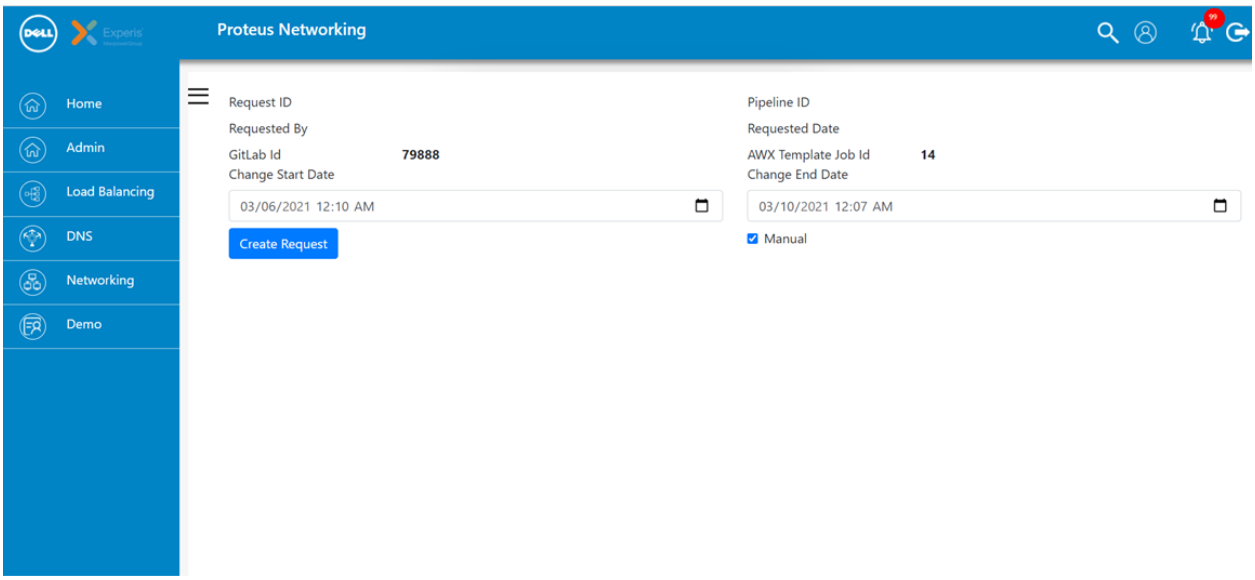


14 Standard Changes with Specific Schedule

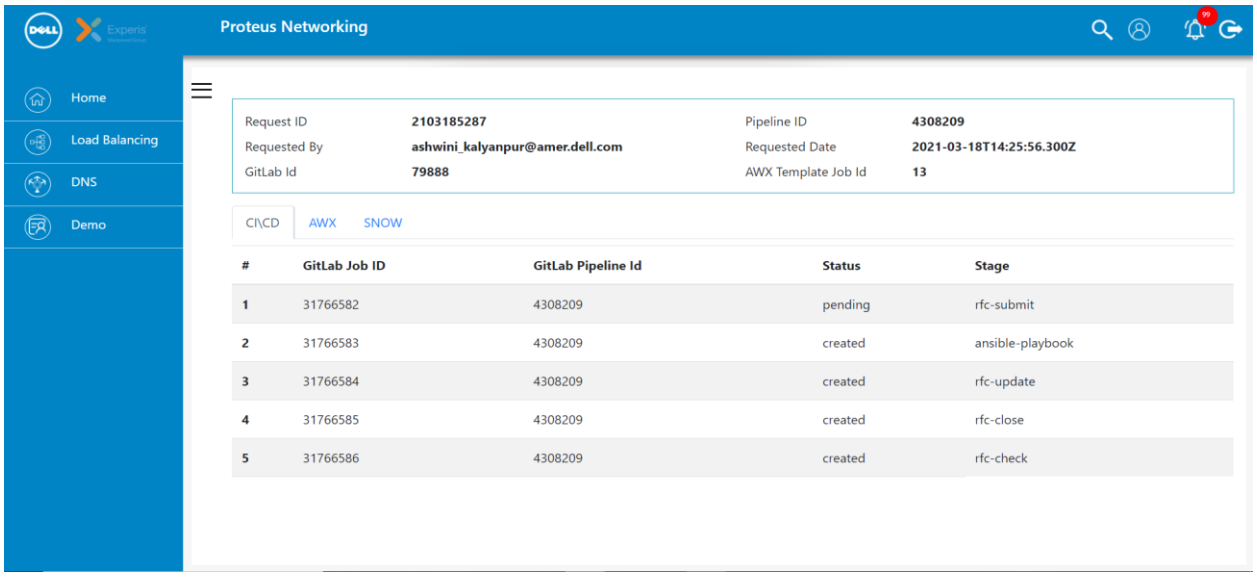
The standard change with specific schedule allows the user to schedule a change with specific time schedule.



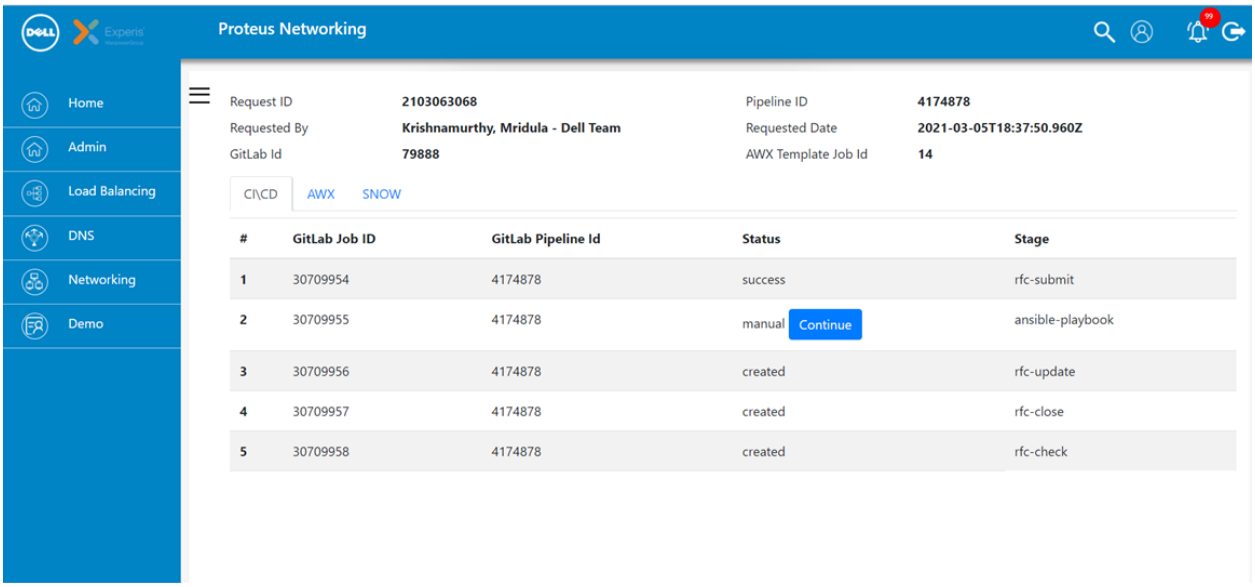
14.1 Create a Change Request



14.2 Ansible Tower Schedules and Executes the Job

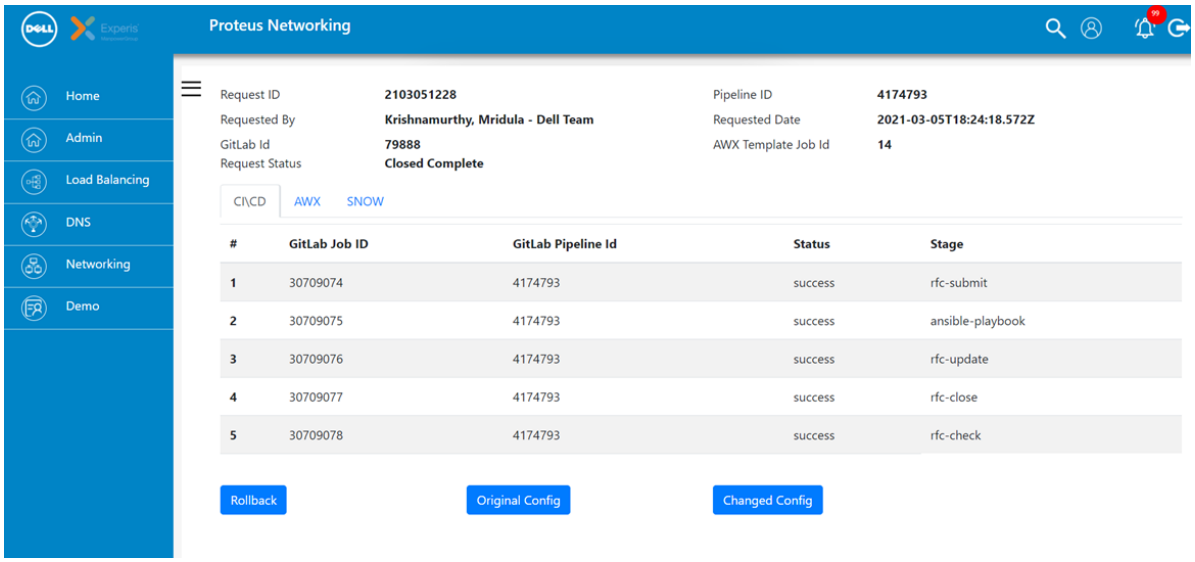


14.3 Review Request and Approve RFC

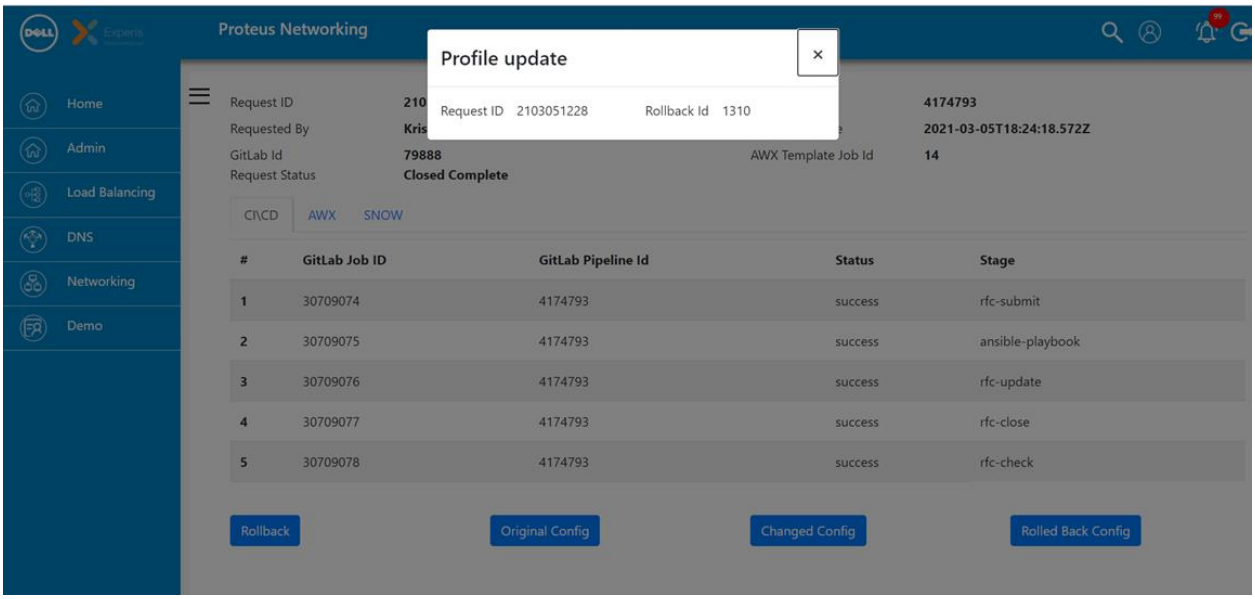


*Rollback Button Appears after the RFC is Closed Complete in ServiceNow.

14.4 Rollback to Original Configuration



14.5 Rollback ID for Reviewing Original and Changed Config



15 System Security and Integrity Controls

15.1 Role-based AD Authentication

To separate the duties of individuals and to reduce the risk of malevolent activity without collusion, each module access is based on role-based AD authentication.

15.2 Security Testing

These testing tools are used to perform security testing:

- Code Quality Scan Software Standard (Sonar Cube)
- Security Scan for software Standard (Checkmarx)
- 3rd Party Code Scan Software Standard (Blackduck: To check security vulnerabilities)
- Hashicorp Vault (To store credentials of SA/DB account)
- TwistLock (To Check the Integrity of Docker Image)

16 Appendix